Assignment: Comparative Study — Multi-Class Wine Quality Classification

Datasets (use BOTH)

- winequality-red.csv
- winequality-white.csv

Section 1 — Problem Statement + Data Description

Problem Statement

You are given two tabular datasets containing lab-measured properties of wines. Your goal is to build a **multi-class classification** model that predicts the wine's **quality score** and then conduct a **comparative study**:

1. **Red vs White dataset:** Which dataset is easier to predict and why?
2. **Model vs Model:** Which model performs better on each dataset and why?
3. **Class-level difficulty:** Which quality scores are hardest to predict?

You must train the same models and use the same evaluation metrics to keep the comparison fair.

Data Description

What one row represents

Each row is **one wine sample** with measured chemical/physical properties.

Target column

- **quality**: an integer **quality rating** assigned to the wine.
- For this assignment, treat each unique value of quality as a **separate class** (multi-class classification).

Feature columns (all numeric)

© 2025 HeroX Private Limited. All rights reserved

These are common wine chemistry measurements:

- **fixed acidity**: stable acids in wine
- **volatile acidity**: acetic acid (high values can reduce quality)
- **citric acid**: adds freshness
- **residual sugar**: sugar left after fermentation
- **chlorides**: salt content
- **free sulfur dioxide / total sulfur dioxide**: preservatives (SO_2 levels)
- **density**: related to alcohol and sugar
- **pH**: acidity level
- **sulphates**: stability/preservation
- **alcohol**: alcohol percentage

What to expect in this dataset

- **Class imbalance:** some quality scores appear much more frequently than others.
- **Overlapping classes:** quality levels like 5 and 6 may have similar feature values, making them harder to separate.

✅ **Deliverable (in your notebook):** Copy this Section 1 into your notebook (or rewrite it in your own words in ~8–12 lines).

Task 1 — Load & Inspect the Data (Red + White)

Instructions

1. Load both CSV files using `sep=";"`.
2. Print for each dataset:
   - shape (rows, cols)
   - first 5 rows
   - column list
3. Verify the target column `quality` exists.

Deliverables

- Output showing both datasets loaded correctly.
- 2–3 lines confirming the target column and number of features.

© 2025 HeroX Private Limited. All rights reserved

Task 2 — Data Quality Checks + Class Distribution (Comparative)

Instructions (do for red and white separately)

1. Check missing values per column.
2. Check duplicates.
3. Show quality class distribution:
   - counts
   - percentages
4. Plot a bar chart of quality counts.

Comparative Questions (answer in 4–6 lines)

- Which dataset is more imbalanced?
- Which quality classes are rare?
- Why might rare classes be harder to predict?

Deliverables

- Missing values + duplicates output for both datasets
- 2 bar charts (red + white)
- Written comparison answers

Task 3 — Prepare Data for Modeling (Same Setup for Fair Comparison)

Instructions (do for both datasets)

1. Create:
   - X = all columns except quality
   - y = quality
2. Train-test split with:
   - test_size=0.2
   - random_state=42
   - stratify=y

Deliverables

© 2025 HeroX Private Limited. All rights reserved

- Train/test shapes for red and white
- 1–2 lines confirming stratification preserved similar class distribution

---

Task 4 — Model A (Baseline): Logistic Regression (Red vs White)

Instructions

Train **Model A** on **both datasets** using a Pipeline:

- StandardScaler()
- LogisticRegression(max_iter=5000)

Evaluate on test set (must include)

- **Accuracy**
- **Confusion Matrix**
- **Classification Report**
- **F1-score** (use **macro F1-score** consistently and mention that)

Deliverables

- Results for red + results for white
- 4–6 lines answering:
  - Which dataset performed better for Logistic Regression?
  - Which quality class looks hardest (lowest recall)?

Task 5 — Model B (Tree Model): Random Forest (Red vs White)

Instructions

Train **Model B** on both datasets:

- RandomForestClassifier(n_estimators=300, random_state=42)

Evaluate using the same metrics as Task 4:

© 2025 HeroX Private Limited. All rights reserved

- Accuracy
- Confusion Matrix
- Classification Report
- Macro F1-score

Deliverables

- Results for red + results for white
- 4–6 lines answering:
  - Did Random Forest improve compared to Logistic Regression?
  - Which class is still hardest?

Task 6 — Comparative Metrics Summary (Core Comparative Study Output)

Instructions

Create a clean summary for **all four experiments** using only:

- **Accuracy**
- **Confusion Matrix**
- **Classification Report**
- **F1-score**

Required comparison table

Dataset	Model	Accuracy	F1-score	Key Observation (1 line)
Red	Logistic Regression			
Red	Random Forest			
White	Logistic Regression			
White	Random Forest			

Required supporting outputs

For each of the four experiments, include:

- Confusion matrix
- Classification report

© 2025 HeroX Private Limited. All rights reserved

Deliverables

- Completed table
- All four confusion matrices
- All four classification reports
- 4–6 lines summarizing: best dataset + best model + reason

Task 7 — Final Conclusion (Comparative Summary)

Write a final paragraph (10–12 lines) answering:

1. Which dataset was easier to predict (red or white) and why?
2. Which model performed better overall and why?
3. Which quality classes were most confusing and why?
4. What ONE improvement would you try next (choose one):
    - class_weight="balanced"
    - convert to 3 classes (Low/Med/High)
    - hyperparameter tuning
    - feature engineering

Deliverables

- Final conclusion paragraph

Submission Instructions (Required)

Submit **one Jupyter Notebook (.ipynb)** named:
Wine_Quality_Comparative_Study_<YourName>_<YourBatch>.ipynb

Your notebook must include, in order:

1. **Section 1: Problem Statement + Data Description**
2. Task 1 to Task 7 headings (use the same task titles)
3. Outputs/plots shown (do not hide outputs)
4. The completed comparison table (Task 6)
5. Final conclusion (Task 7)

© 2025 HeroX Private Limited. All rights reserved

Formatting rules

- Keep code clean and readable (meaningful variable names, comments where needed).
- Ensure every task has both:
  - **code**, and
  - **short written answers** where asked.
- Do not change the split settings (test_size=0.2, random_state=42, stratify=y) to keep comparisons fair.

Task-Based Grading Rubric (50 points)

Task	What is evaluated	Points
Task 1	Correct loading, basic inspection for both datasets	7
Task 2	Data checks, class distribution + charts, comparison answers	15
Task 3	Correct X/y, stratified split, proper reporting	7
Task 4	Logistic Regression pipeline + required metrics + interpretation	7
Task 5	Random Forest training + required metrics + interpretation	7
Task 6	Comparison table + all confusion matrices + all classification reports + summary	4
Task 7	Final comparative conclusion + next-step proposal	3
Total		**50**

© 2025 HeroX Private Limited. All rights reserved